

A face-orientation estimation system

ACQ23AHT

University of Sheffield

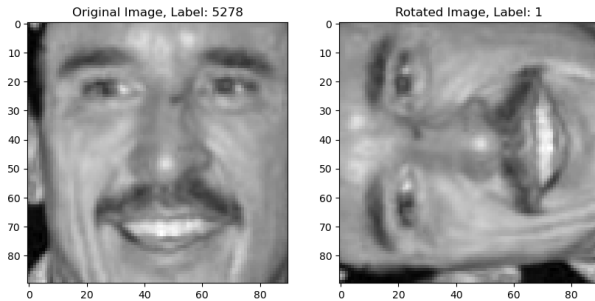


Figure 1: Rotating 90x90 pixel Image Anti-clockwise: 1

Abstract

Machine learning classifiers, like Support Vector Machines (SVM) and k-Nearest Neighbours (KNN), have become versatile tools with a wide range of applications across multiple domains. By using algorithms to examine patterns and relationships in the training data, it is able to apply previously learned knowledge to make well-informed predictions or decisions about new, unseen instances. Pre-processing the data, extracting features, and configuring the classifier hyper-parameters to get the best possible results are important components for enhancing data quality, ensuring it meets the classifier's requirements and influencing the model's accuracy and effectiveness.

1. Introduction

In this report we will build a face classification system using Scikit Learn to build a face rotation classification system. We have two joblib files, one small and the other full, each contain a list of arrays that represent an image in an upright orientation. The small and full joblib contains 2500 and 10803 images respectively, each of 125 by 94 pixels, labelled with the person-id.

A patch extractor is used to split the images into sub-images, and for each sub-image, a separate training model is applied for the pixel groups 30, 50, and 90. The development of specific scikit-learn models is done by fine tuning the hyper-parameters and choosing the best fit. To attain the highest accuracy in our classification, we can select from a variety of classifiers, including K-Nearest Neighbours, Support Vector Machine, Random Forest (RF), Neural Network Classifier, and others. The project's goal is to demonstrate how well machine learning methods—in particular, scikit-learn—can classify facial image orientations accurately using extracted sub-regions. The models' practical applicability for face orientation determination in real-world scenarios will be ensured by rigorous testing and validation, which will assess the models' effectiveness.

2. System Description

We create our `train.py` in such a way that we can pass the input file and the required `model.joblib` as a parameter. The two input joblibs can be extracted in the form of images and person id by using the `load` function from the `joblib` library. The images consists of the dimensions (2500,125,94) and the labels contains (2500,). A tool for extracting fixed-size patches from images and facilitating feature extraction for machine learning tasks is the `PatchExtractor` class in `scikit-learn`. It helps create informative input data for image-based models by letting us set parameters like the maximum number of patches, patch size, and more. To do so, we pass one of the arguments called `n` pixels as a parameter for the function `patch` and save the return output in sub images.

The sub images are passed as an input to the function `rotate` images where each image is rotated, using the `numpy` `rot90` function, randomly by 0 - upright, 1 - rotate-left, 2 - upside down and 3 - rotate-right. The rotated images are reshaped and converted into a `numpy` arrays. These serve as input for the SVM(Support Vector Machine) classifier. Similar to a smart boundary setter, (SVM) is used for image classification. It examines various aspects of an image and attempts to determine the best border or line that can divide various classes, like determining a face's orientation.

The classifier function's `makedirs()` creates a temporary directory, and `Memory(cachedir, verbose=2)` uses this directory to cache and speed up computations, enhancing the efficiency of memory-intensive tasks in Python programs. The input is split into 80 percent training and 20 testing data. The model is represented as a pipeline of operations, including scaling the data using `Standard Scaler`, dimensionality reduction using `PCA` and `SVC` with specific hyper-parameters. The function returns the model trained by the classifier and by using `joblib` `dump` function, we create the `joblib` named as one of the input arguments to `train.py`. Hence, having created the `model.joblib`, we can run the `evaluate.py` to test the accuracy of the built model.

3. Experiments

First, we use the '`train.small.joblib`' dataset, which has 2,500 images with a pixel size of (125,94), to train our model. This will enable us to apply the model to the entire input file and obtain a clear understanding of which parameters are best for maximum accuracy. The function `patch` uses the input parameter, `n` pixels, to extract the particular pixel dimensions from the input images. The images are split 80 and 20 percent into train and test respectively. To tackle the classifier, we use the Scikit-learn's method known as `GridSearchCV`. It is used for fine-tuning hyper-parameters in machine learning models. Using cross-validation, it iteratively searches through a predefined hyper-parameter grid to assess the model's performance. The model is then optimised for a particular dataset by choosing the set of hyper-parameters that produces the best results.

Therefore, by using `GridSearchCV`, we train the classifiers,

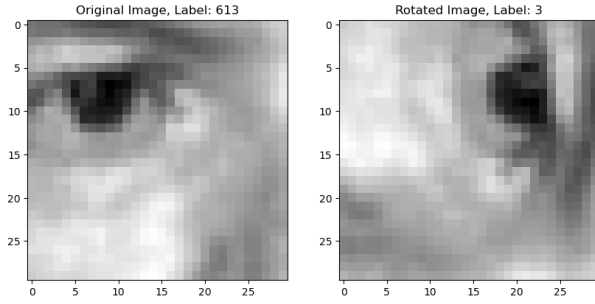


Figure 2: Rotating 30x30 pixel Image Anti-clockwise: 3

k-nearest neighbors, random forest and support vector machine. KNN is a straightforward and efficient method for classifying images since it uses the majority class of the closest neighbours of the image in the feature space to classify the images. By iterating over the range of values of the n neighbours and PCA's n components, GridSearchCV fine-tunes the parameters. We find that values 9 and 50, respectively, have a comparably greater accuracy than others. Similarly, A collaborative learning algorithm called Random Forest is used to classify images. In order to increase accuracy and avoid overfitting, it constructs several decision trees during training and combines their predictions. After running the GridSearchCV on this classifier a couple of times we observe an average accuracy output to each of the pixel images. As a result, we employ the SVM classifier and utilise pipelining to incorporate additional techniques.

We use the pipeline method to first normalise the input so that it can help in making different features more comparable. This is followed by PCA, where each value of n components is tested over a range of specified values. The kind of decision boundary that the SVM will employ is decided by several hyper-parameters namely, kernel - determines the type of decision boundary, C - regularisation parameter that controls the trade-off between achieving a smooth decision boundary and correctly classifying the training points, gamma - defines how far the influence of a single training example reaches. Following a range of parameters for accuracy testing, we find that the best results are obtained when kernel= rbf, $C=2$, and gamma=scale.

Finally after training and testing the classifier model, we use the 'dump' function in joblib to create a joblib file in the same folder. For more effective storage, this file is compressed which makes sure it is under 20MB. After training the input images and testing the hyper-parameters on the small joblib, we continue training the model on the full.joblib consisting of 10,803 images with dimensions (125,94) pixels. The increase in the size of the input dataset plays a key role in calculating the accuracy of the model as the number of training and testing examples have increased. This makes the model more reliable and efficient.

4. Results and Analysis

From the table below we can determine that after running the evaluate.py on the trained models, we get a calculated approach towards the optimal value of PCA which is 50 for 90 pixels and 40 for 50 and 30 pixels. This training on the small joblib allows us to train our model in a specific way on the full joblib to achieve maximum accuracy.

The table shows that the combined application PCA and GridSearchCV plays an important role in determining the op-

Table 1: Accuracy after training the SVM Classifier on train.small.joblib

Pixels	PCA	% Correct
90	50	91.45
90	50	91.2
50	70	55.1
50	40	56.8
30	40	39.3
30	20	37.7

timal hyper-parameters for the classifier. The table shows the accuracy fluctuations due to the changing values of PCA n components.

Table 2: Accuracy after training the SVM Classifier on train.full.joblib

Pixels	PCA	% Correct
90	50	95.39
50	40	64.6
30	40	43.5

5. Discussion and Conclusions

We can observe from the results that the SVM classifier worked most efficiently for the face classification challenge. It was able to provide excellent results with appropriate values of PCA n components. Thus after applying the key values by testing the model on the small joblib we were able to improve our model greatly. We could find good improvements after performing hyper-parameter tuning. With some more time I would like to test and go deeper testing other classifiers and find how the accuracy changes with tuning.

6. References